

Especificação — Inserção de "Specification Engineering" no pipeline do LangNet

Autor: Claude (assistente) · **Data:** 2026-08-12 · **Status:** proposta detalhada para aprovação **Base conceitual:** artigo "Specification Engineering: The New Skill After Prompt Engineering" (KDnuggets, ago/2026) **Alvo:** o gerador de apps agênticos do LangNet (backend/agents/langnetagents.py) + etapas do pipeline

0. Contexto e motivação

0.1 O que o artigo defende

A tese central: a habilidade que sucede *prompt engineering* é **specification engineering** — em vez de "pedir melhor", **definir** de forma executável, testável e auditável: 1. **Objetivo** — o que o modelo deve alcançar; 2. **Contexto** — o que ele precisa saber; 3. **Inputs** — que dados/arquivos/ferramentas/assunções são permitidos; 4. **Formato de saída** — como a resposta final deve ser; 5. **Restrições** — o que evitar; 6. **Critérios de avaliação** — como julgar correção; 7. **Edge cases** — o que pode dar errado; 8. **Passos de verificação** — que testes/checagens devem passar.

E propõe um **workflow** que substitui `prompt` → `output` → `conserto manual` por:

especificação → geração → validação → revisão (só do que falhou) → auditoria (registrar suposições/limitações)

Dois conceitos do artigo são especialmente relevantes para nós: - **Structured Outputs** (OpenAI): "specification engineering em forma de API" — em vez de *torcer* para o modelo devolver JSON válido, o desenvolvedor **define o schema que a saída é obrigada a obedecer**. - **Specification gaming** — o sistema satisfaz o objetivo *escrito* mas erra o *pretendido* ("o prompt funcionou; a especificação falhou").

0.2 Por que isso importa PARA NÓS agora (evidência do nosso próprio código)

Nas últimas rodadas de trabalho no gerador, **toda uma família de bugs** teve a mesma raiz: a **saída do agente não tem contrato** — o ws-server manda o que vier, e a persistência quebra ou aceita lixo. Exemplos reais que corrigimos por **reparo** (band-aid), não por **contrato**:

Sintoma real	Onde apareceu	Reparo atual (band-aid)
Resultado do agente vem embrulhado em { "raw": "...json..." }	<code>_execute_task</code> (ws-server), linha ~2695: <code>parsed = [{"raw": raw}]</code>	<code>parseAgentResult</code> no frontend (<code>_AGENT_BODY</code>) desembulha
<code>nivel_confianca</code> é FLOAT no schema mas o agente devolve enum <code>"alta"</code>	<code>criar_pre_diagnostics</code>	helper <code>_cv</code> coage <code>'alta'→0.9</code> no adapter
<code>hipoteses</code> vem como objeto/lista mas a coluna é TEXT	idem	<code>_cv</code> faz <code>json.dumps</code>
Agente devolve sem <code>hipoteses</code> / <code>nivel_confianca</code> → <code>INSERT</code> falha por <code>NOT NULL</code>	pré-diagnóstico/prontuário	best-effort silencioso (a etapa "passava" mas nada persistia)
Task consulta tabela inexistente (<code>historico_medico</code>)	<code>pre_atendimento_*</code>	

Sintoma real	Onde apareceu	Reparo atual (band-aid)
		guard _validate_tasks_schema_coherence + nota de coerência

O padrão é claro: **remendamos a saída depois que ela já chegou torta**. O artigo aponta o caminho principiado: **derivar um contrato de saída e validá-lo/repará-lo na fonte** — e, quando o contrato não puder ser satisfeito, **falhar em voz alta** (fail-loud) em vez de persistir algo incompleto (specification gaming).

0.3 O que já temos alinhado ao artigo (para não reinventar)

- **Decomposição em passos** (o "quebre recursos densos em passos menores"): decomposição de tarefas + Rede de Petri.
- **Spec parcial por task**: tasks.yaml já tem, por task, description com Input format + Process steps e um expected_output (em **prosa**).
- **Versão/refino/aprovar por etapa** (spec→geração→revisão), com histórico versionado.
- **Uma validação estrutural**: o guard de coerência tasks↔schema (_validate_tasks_schema_coherence / _annotate_tasks_coherence) — já é "specification engineering" aplicado (valida referências).
- **Reparos de saída**: parseAgentResult (frontend) e _cv (adapters).

0.4 Princípio-guia desta proposta

Trocar **reparo pós-fato** por **contrato na fonte**: cada task agêntica ganha um **contrato de saída** derivado do expected_output e das restrições reais do banco; a saída é validada/reparada **no ws-server** antes de persistir; o que não satisfizer o contrato **falha explicitamente**.

1. Mapa: os 8 elementos da especificação × onde vivem no nosso pipeline

Nosso pipeline: **Requisitos** → **Especificação** → **Modelo de Dados** → **Casos de Teste** → **Sequência de Tarefas** → **Rede de Petri** → **Código** (agents.yaml + tasks.yaml + adapters + ws-server + telas).

#	Elemento da spec	Onde EXISTE hoje	Lacuna
1	Objetivo	tasks.yaml → description (1ª linha)	OK
2	Contexto	Especificação (UC) + agents.yaml (backstory)	OK
3	Inputs	description → "Input format" + *_input_func	OK (mas não validado)
4	Formato de saída	expected_output em prosa	Sem schema executável ⇒ Inserção A
5	Restrições	esparso na description	fraca ⇒ Inserção C
6	Critérios de avaliação	Casos de Teste (CEG) — desconectado do runtime	⇒ Inserção B
7	Edge cases	fallback_manual + pouco mais	fraca ⇒ Inserção C
8	Passos de verificação	guard de coerência (parcial)	Sem pós-condições por task ⇒ Inserção B

As duas maiores lacunas — **#4 (saída sem schema)** e **#8 (sem verificação por task)** — são exatamente as Inserções **A** e **B**, e são as que atacam os bugs reais.

2. Inserção A — Contrato de saída (JSON Schema) por task agêntica + validação/reparo no ws-server

É a peça central. Implementa "Structured Outputs" do artigo dentro do LangNet.

2.1 O que é

Cada task **agêntica** (que roda no CrewAI, não os adapters determinísticos) passa a ter um **contrato de saída** — um **JSON Schema** com: campos, tipos, `required`, e `enum`/domínios quando aplicável. Esse contrato é: 1. **Gerado** a partir de duas fontes combinadas: - o `expected_output` da task (o que a spec *declara*); e - as **colunas NOT NULL da entidade** que a task persiste (o que o banco *exige*). 2. **Anexado** à task (campo `output_schema`: no `tasks.yaml`, ou artefato irmão). 3. **Aplicado** no ws-server: a resposta do agente é **normalizada** → **coagida** → **validada** contra o schema antes de virar `task_completed`. Falta de campo obrigatório ⇒ **erro explícito**.

2.2 Por que é importante (o problema que resolve, com precisão)

Hoje, no template do ws-server (`_template_websocket_server.py`, função `_execute_task`), o fluxo é:

```
# backend/agents/langnetagents.py (dentro do template do ws-server)
raw = getattr(result, "raw", None) or str(result) # ~linha 2686
output_fn = getattr(adapters_module, f"{task_name}_output_func", None)
if callable(output_fn):
    parsed = output_fn(input_data, raw)
else:
    parsed = {"raw": raw} # ~linha 2695 ← ORIGEM do {raw:...}
await _send(ws, "task_completed", {"task_name": task_name, "result": parsed}) # ~linha 2697
```

Ou seja: **sem `output_func`, manda-se `{raw: <texto do agente>}` cru** — daí o frontend precisar do `parseAgentResult`, e a persistência precisar do `_cv`, e ainda assim campos faltando geram falha `NOT NULL` silenciosa. **A validação de contrato entra exatamente entre a linha 2695 e a 2697** e: - desfaz o envelope `{raw}` /string → objeto (**elimina** a razão de existir do `parseAgentResult`); - coage tipos ao domínio do schema (**absorve** o papel do `_cv`, agora na fonte e para toda task); - **rejeita** saída que não cumpre `required` — em vez de deixar o INSERT falhar lá na frente (combate ao *specification gaming*: "o agente respondeu algo, mas não o que o contrato pede").

2.3 Onde entra (arquivos, funções, pontos de inserção)

(a) Geração do schema — no gerador, junto de onde o `tasks.yaml/adapters` são montados - Arquivo:

`backend/agents/langnetagents.py`. - **Nova função:** `_derive_output_schema(task_name, task_cfg, entity_model) -> dict`. - Parseia o `expected_output` (prosa) em campos+tipos. Ex. real de `pre_atendimento_cardiologia`: `expected_output`: - hipoteses: JSON contendo as possíveis condições... - `nivel_confianca`: Enum (baixa/média/alta) - `exames_sugeridos`: Texto... → `{hipoteses: object|array, nivel_confianca: enum[baixa,média,alta], exames_sugeridos: string}`. - **Cruza com a entidade persistida** (via a mesma lógica de `RESULT_FK / SAVE_ENTITY / CHAIN` que já temos): para `pre_diagnostics` o schema marca `required: [hipoteses, nivel_confianca]` (as colunas `NOT NULL` não-FK), com **tipo do banco** (`nivel_confianca: number` porque a coluna é `FLOAT`, mesmo que a prosa diga "Enum") — resolvendo o conflito `enum↔float` **no schema**, não no adapter. - Reaproveita utilitários existentes: `_schema_model` (colunas/tipos/`NOT NULL`), o parser de tipos do `_cv`, e o parser de campos de `expected_output`. - **Emissão:** anexa `output_schema`: a cada task agêntica no `tasks.yaml` gerado (bloco YAML), da mesma forma que hoje anexamos a nota de coerência (`_annotate_tasks_coherence`) e como o guard já reescreve o `tasks.yaml`. Alternativa: um arquivo `ws-server/output_schemas.json` carregado pelo ws-server.

(b) Aplicação — no template do ws-server - Arquivo/local: `_template_websocket_server.py`, dentro de `_execute_task`, **entre as linhas ~2695 e ~2697**. - **Nova função no template:**

`_coerce_to_schema(raw, schema) -> (obj, faltantes)` que: 1. **desembrulha**: se `raw/parsed` é `{raw: "..."} ou string` com cerca markdown, extrai o JSON real (mesma lógica do `parseAgentResult`, agora no servidor); 2. **coage por campo** conforme o `type` do schema (enum textual→número via mapa `baixa/média/alta`, `dict/list`→JSON string p/ TEXT, `"70%"`→`0.7`, datas ausentes→hoje) — a lógica do `_cv`, aplicada a **toda** saída agêntica; 3. **valida required** e devolve os campos faltantes. - **Decisão de envio**: - se cumpre o contrato → `task_completed` com o objeto **já normalizado** (o frontend recebe objeto limpo; `parseAgentResult` vira redundante e pode ser removido depois); - se falta obrigatório → **1 retry** com o schema reforçado no prompt (o próprio artigo: "revisar só contra a checagem que falhou"); persistindo a falha → `error` explícito `"saída não cumpre o contrato: faltam [hipoteses]"` (fail-loud), **sem** `task_completed`.

(c) Onde o schema é lido no ws-server - O `ws-server` já carrega `TASKS_CONFIG` do `tasks.yaml`. Passa a expor `TASKS_CONFIG[task]['output_schema']` (ou o `output_schemas.json`). Nenhuma mudança de protocolo com o frontend.

2.4 Exemplo concreto (antes/depois) — `pre_atendimento_cardiologia`

Schema derivado (novo `output_schema`):

```
{
  "type": "object",
  "required": ["hipoteses", "nivel_confianca"],
  "properties": {
    "hipoteses": { "type": ["object", "array", "string"] },
    "nivel_confianca": { "type": "number", "coerce_from_enum": { "baixa": 0.4, "média": 0.7, "alta": 0.9 } },
    "exames_sugeridos": { "type": "string" }
  }
}
```

Resposta crua do agente (o que hoje quebra):

```
{ "hipoteses": {"angina": 0.8}, "nivel_confianca": "alta", "exames_sugeridos": "ECG, troponina" }
```

- **Hoje**: vira `{raw:"..."}` → frontend desembulha → `_cv` conserta `nivel_confianca` → às vezes ok, às vezes o agente omite `hipoteses` e o INSERT falha silencioso.
- **Com a Inserção A**: o `ws-server` valida na fonte → `nivel_confianca: 0.9`, `hipoteses` vira JSON string, `required` satisfeito → `task_completed` com objeto limpo. Se o agente omitisse `hipoteses`, o `ws-server` **rejeitaria** (fail-loud), sem gravar meio-registro.

2.5 Arquivos tocados / esforço / risco

- **Tocados**: `backend/agents/langnetagents.py` (nova função de derivação + injeção no `tasks.yaml` + bloco no template do `ws-server`). Opcional: gerar `ws-server/output_schemas.json`.
- **Esforço**: médio (2 funções novas + 1 bloco no template). Reaproveita `_schema_model`, `_cv`, parser de `expected_output`.
- **Risco**: baixo/médio. Mitigação: **modo tolerante por padrão** (coage e loga) e **fail-loud opcional** por task (para não travar apps já existentes); o guard só rejeita quando `required` não é satisfeito.
- **Verificação**: regenerar ClinIA; rodar as tasks agênticas (triagem, pré-diagnóstico, consulta) e conferir que (i) o frontend recebe objeto limpo (sem `{raw}`), (ii) `nivel_confianca` chega numérico, (iii) uma resposta propositalmente incompleta gera `error` explícito e **não** persiste.

2.6 O que a Inserção A aposenta (dívida técnica removida)

- `parseAgentResult` (frontend) — passa a ser redundante (a saída já chega normalizada).
- `_cv` disperso — a coerção passa a ser **guiada pelo schema** e centralizada no `ws-server`.

3. Inserção B — Checagens de verificação (pós-condições) por task + "revisar só o que falhou"

Implementa os elementos **#6 (avaliação)** e **#8 (verificação)** e os passos 4–5 do workflow do artigo.

3.1 O que é

Cada task ganha uma seção **declarativa** `verification:` com pós-condições checadas **após** a execução (determinística ou agêntica). Ex.:

```
criar_encaminhamento:
  verification:
    - not_null: [atendimento_id, especialidade_id, medico_id] # FKs obrigatórias preenchidas
    - row_created: encaminhamentos # a linha existe no banco
registrar_prontuario:
  verification:
    - not_null: [pre_diagnostico_id, encaminhamento_id]
    - fk_matches_current_attendance: [atendimento_id] # liga ao atendimento corrente
```

3.2 Por que é importante

- Transforma os **Casos de Teste** (que hoje vivem só na etapa CEG, desconectados) em **checagens de runtime** — o artigo insiste que "specification define se a resposta é *aceitável*", não só plausível.
- Dá base ao **"revise only against failed checks"**: quando um check falha, alimentamos **apenas o check falho** de volta ao agente/refino (mais barato e focado que refazer tudo). Isso encaixa no nosso mecanismo de refino por chat já existente.
- Detecta *specification gaming* concreto: "o encaminhamento foi criado, mas com `medico_id` nulo" — hoje isso passava; com a pós-condição `not_null`, vira erro.

3.3 Onde entra

- **Geração**: derivar `verification:` automaticamente das **restrições do schema** (colunas NOT NULL/FK da entidade da task) — reaproveitando `_schema_model` e a lógica de FK que já usamos na Inserção do item 3. Arquivo: `backend/agents/langnetagents.py`.
- **Execução**: no template do ws-server (`_execute_task`), **após** o `det_fn`/agente retornar (linha ~2631 para determinístico e ~2697 para agêntico), rodar `_run_verifications(task, result, input_data)`; falha → `error` com a lista de checks reprovados (ou marca `warnings` no `task_completed`, configurável).
- **Refino dirigido por check**: endpoint de refino das tasks (`app/routers/tasks_yaml.py`, `POST /{sid}/refine`) ganha a opção de receber "corrija a task para satisfazer o check X" — fechando o laço `validação→revisão`.

3.4 Esforço / risco

- **Esforço**: médio-alto (linguagem declarativa de checks + executor + integração com refino).
 - **Risco**: médio. Mitigação: começar com um conjunto pequeno de checks (`not_null`, `row_created`, `output_has`) e modo *warning* antes de *fail*.
 - **Dependência**: reaproveita o schema/entidade da **Inserção A** (fazer A antes de B).
-

4. Inserção C — Template dos 8 elementos + passo "identificar requisitos faltantes"

Ataca a **qualidade upstream** da especificação (o artigo cita o estudo ROPE: +20% de qualidade quando o requisito é bem articulado; e "AI amplifica forças/fraquezas do processo" — DORA).

4.1 O que é

1. **Checklist dos 8 elementos** imposto na geração da **Especificação** (por caso de uso) e do **Agent-Task Spec** (por task): objetivo, contexto, inputs, **output**, **constraints**, **evaluation**, **edge cases**, **verification**. Onde o gerador hoje produz `description + expected_output`, passa a produzir também `constraints:` e `edge_cases:` explícitos.
2. **Passo "gap analysis"**: antes de gerar a solução, um passo do agente que **lista requisitos faltantes/ambiguidades/assunções** — o passo 2 do workflow do artigo ("Ask the AI to identify missing requirements").

4.2 Onde entra

- **Prompts de geração:** `backend/app/routers/specification.py` (prompts da Especificação) e `backend/app/routers/agent_task_spec.py` (Agent-Task Spec) — acrescentar as seções obrigatórias e a instrução de gap-analysis.
- **Persistência:** as seções novas entram no documento versionado da etapa (sem mudar o schema de sessões).

4.3 Por que é importante

- `constraints / edge_cases` explícitos reduzem *specification gaming* (o artigo: "Do not claim causality", "Do not use external paid APIs" como exemplos de restrições que mudam o resultado).
- O passo de gap-analysis materializa "a qualidade do input determina a qualidade do output".

4.4 Esforço / risco

- **Esforço:** baixo-médio (mudança de prompts + uma seção nova por artefato).
- **Risco:** baixo. É aditivo; não muda runtime.

5. Inserção D — Log de suposições & limitações (auditoria)

Passo 6 do workflow ("Log the final assumptions and limitations").

5.1 O que é / onde entra

Cada etapa de geração passa a registrar um bloco `assumptions_and_limitations` (o que assumiu, o que ficou fora de escopo, riscos conhecidos). Estende a **proveniência** que já gravamos (migrations 023–029) com mais uma coluna/campo, exibido na UI ao lado do "Origem: ...". - **Arquivos:** os routers de cada etapa (gravam o campo) + a UI da etapa (exibe).

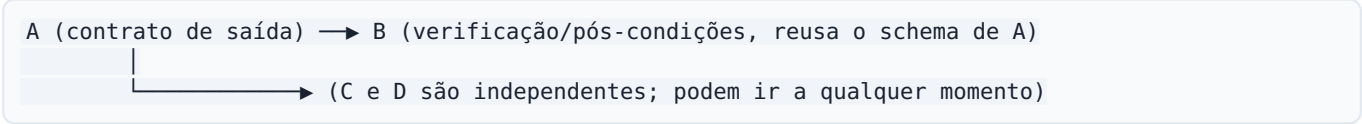
5.2 Por que é importante

Auditabilidade: transforma suposições implícitas em registro explícito e rastreável — coerente com a proveniência versionada que já temos.

5.3 Esforço / risco

- **Esforço:** baixo (campo + exibição). **Risco:** baixo (aditivo).

6. Ordem de implementação recomendada e dependências



1. **A** primeiro — maior impacto, remove a família de bugs de saída, e é pré-requisito natural de B.
2. **B** — fecha o laço validação→revisão.
3. **C e D** — melhorias de qualidade/auditoria, baratas, aditivas.

Sugestão de MVP: implementar **A** de ponta a ponta na ClinIA (gerar `output_schema`, validar no ws-server, provar que o `{raw}` some e que uma saída incompleta falha em voz alta), medir, e então decidir sobre B/C/D.

7. Impacto esperado — cada bug que remendamos × como a proposta previne na fonte

Bug remendado	Reparo atual	Prevenção com A/B
<code>{raw:...}</code> no frontend	<code>parseAgentResult</code>	A normaliza no ws-server; frontend recebe objeto limpo
enum <code>"alta"</code> em coluna <code>FLOAT</code>	<code>_cv</code> no adapter	A coage guiado pelo schema (tipo do banco) na fonte
<code>dict</code> em coluna <code>TEXT</code>	<code>_cv json.dumps</code>	A idem
campo obrigatório faltando → <code>NOT NULL</code>	best-effort silencioso	A rejeita (fail-loud) + B pós-condição <code>not_null</code>
FK nula persistida (ex.: <code>medico_id</code>)	— (só descoberto no banco)	B <code>not_null / row_created</code> barra na hora
task consulta tabela fantasma	guard de coerência (já feito)	mantém; C reforça constraints

8. Riscos gerais e mitigações

- **Rigidez demais** (contrato rejeitando saídas boas por detalhe de tipo): começar em **modo tolerante** (coage+loga), `fail-loud` opt-in por task; `required` só para o que o banco realmente exige (`NOT NULL`).
- **LLM local flaky/lento** (já conhecido): o retry único guiado por schema (A) e por check (B) ajuda, mas não elimina; manter os timeouts atuais.
- **Divergência spec↔schema** (`expected_output` diz X, coluna é Y): a Inserção A **resolve pelo banco** (fonte de verdade da persistência) e o guard de coerência (já existente) sinaliza o resto.
- **Compatibilidade com apps já gerados:** tudo é opt-in/aditivo no gerador; regenerar um app aplica as melhorias sem quebrar o protocolo ws-server↔frontend.

9. Resumo executivo

O artigo formaliza o que nosso pipeline já tentava fazer aos pedaços. A inserção de **maior valor e menor atrito** é o **contrato de saída (Inserção A)**: derivar um JSON Schema por task agêntica (do `expected_output` + das restrições `NOT NULL` /tipo do banco) e **validar/reparar a saída do agente no ws-server** (`_execute_task` , entre as linhas ~2695–2697), com **fail-loud** quando o contrato não é cumprido. Isso troca três band-aids (`parseAgentResult` , `_cv` , best-effort silencioso) por **um contrato na fonte**, e prepara o terreno para **B** (pós-condições/verificação), **C** (spec dos 8 elementos + gap-analysis) e **D** (log de suposições/limitações).